
Assignment 1.

Advanced Quality Assurance

By

VLADISLAV KHEGAY

Group: CSE-2502M



Department of School of Software Engineering
ASTANA IT UNIVERSITY

Submitted to:

Azamat Serek

Contents

1	Introduction	1
1.1	System Description	1
1.2	Objectives	1
2	Main Part	1
2.1	Risk Assessment & Strategy Planning	1
2.1.1	System Analysis	1
2.1.2	Risk Assessment	2
2.2	QA Environment Setup	2
2.3	Initial Test Strategy Documentation	3
2.3.1	Project Scope and Objectives	3
2.3.2	Test Approach	3
2.3.3	Tool Selection and Configuration	3
2.4	Baseline Metrics for Research Paper	4
3	Conclusion	4

1 Introduction

1.1 System Description

GAN-Front is an Angular 18 web application with Server-Side Rendering (SSR) implemented using Angular Universal and Express.js. The application includes authentication, protected routes, performance optimization services, and forensic-themed UI components.

Key modules are located in `src/app/services/` and include:

- `auth.service.ts`
- `hydration-mismatch.service.ts`
- `api.interceptor.ts`
- `server-api.service.ts`
- `performance-optimization.service.ts`
- `image-optimization.service.ts`
- `font-optimization.service.ts`
- `forensic-state.service.ts`
- `forensic-signals.service.ts`

The project can be built and run using `npm run build:ssr` and `npm run serve:ssr`.

1.2 Objectives

- Analyze the chosen system
- Identify critical components or modules
- Prioritize testing based on risk probability vs impact
- Document assumptions and reasoning
- Set up test repository and version control

2 Main Part

2.1 Risk Assessment & Strategy Planning

2.1.1 System Analysis

The system contains the following main modules:

- Authentication Service and Route Guards (`auth.service.ts`)
- SSR Hydration Service (`hydration-mismatch.service.ts`)
- API Interceptor and Server API Service (`api.interceptor.ts`, `server-api.service.ts`)
- Performance Optimization Service (`performance-optimization.service.ts`)

- Image and Font Optimization Services (image-optimization.service.ts, font-optimization.service.ts)
- Forensic State and Signals Services (forensic-state.service.ts, forensic-signals.service.ts)
- Login, Dashboard and Lighthouse Demo components

2.1.2 Risk Assessment

- **Authentication Service + Route Guards** (auth.service.ts) — Impact: High, Probability: High, Risk: **High**. Any failure in token validation or guard resolution directly blocks access to all protected routes.
- **SSR Hydration Mechanism** (hydration-mismatch.service.ts) — Impact: High, Probability: Medium, Risk: **High**. A hydration mismatch between the server-rendered HTML and the client-side Angular bootstrap silently breaks interactivity.
- **API Interceptor Layer** (api.interceptor.ts, server-api.service.ts) — Impact: High, Probability: High, Risk: **High**. Mediates every outbound HTTP request; a defect affects all network-dependent features.
- **Performance Optimization Services** (performance-optimization.service.ts, image-optimization.service.ts, font-optimization.service.ts) — Impact: High, Probability: Medium, Risk: **High**. Regressions directly degrade Core Web Vitals scores (LCP, FID, CLS).
- **Forensic State and Signals Services** (forensic-state.service.ts, forensic-signals.service.ts) — Impact: Medium, Probability: Medium, Risk: **Medium**.
- **UI Components** (Login, Dashboard, Lighthouse Demo) — Impact: Low, Probability: Low, Risk: **Low**.

Assumptions:

- The application operates in a production-like environment.
- Authentication serves as the entry point for protected features.

High-risk modules are prioritized for testing.

2.2 QA Environment Setup

- Test repository: <https://github.com/TheKhegaPlay/GAN>
- Version control: Git
- Available scripts: npm test, npm run lint, npm run build:ssr, npm run serve:ssr

2.3 Initial Test Strategy Documentation

2.3.1 Project Scope and Objectives

The scope of this QA effort covers all critical server-side and client-side modules of the GAN-Front Angular 18 SSR application.

The primary objective is to verify that the authentication flow is secure and reliable, that the SSR hydration process produces no DOM mismatches, and that the API interceptor correctly handles token injection, error responses, and request caching.

Secondary objectives include validating the Performance Optimization Service against Core Web Vitals thresholds ($LCP \leq 2.5\text{ s}$, $FID \leq 100\text{ ms}$, $CLS \leq 0.1$), confirming that the Forensic State and Signals Services maintain accurate application state, and ensuring that UI components render correctly under both SSR and CSR conditions.

Out of scope for this assignment are backend GAN model testing, infrastructure provisioning, and third-party service integrations.

2.3.2 Test Approach

Testing for GAN-Front is organized entirely around the CI/CD pipeline. All quality checks are triggered automatically by GitHub Actions on every push and pull request targeting the main branch.

The pipeline executes the following stages in order:

- **Lint stage** — `npm run lint` runs ESLint with `@angular-eslint`.
- **Build stage** — `npm run build:ssr` compiles the Angular 18 application with SSR enabled.
- **Unit & Integration test stage** — `npm test` executes the test suite.
- **Performance audit stage** — Lighthouse CLI runs in headless mode.

Each stage depends on the success of the previous one. A failure in any stage blocks merging of the pull request.

2.3.3 Tool Selection and Configuration

All tools are integrated into the GitHub Actions pipeline:

- **GitHub Actions** — Orchestrates the full pipeline via `.github/workflows/ci.yml`.
- **ESLint with @angular-eslint**
- **Jest** (or Karma/Jasmine — depending on your actual setup)
- **Playwright** — for E2E tests
- **Lighthouse CLI** — for performance auditing

2.4 Baseline Metrics for Research Paper

- **Count of high-risk modules:** 4 (Authentication Service & Route Guards, SSR Hydration Mechanism, API Interceptor Layer, Performance Optimization Services).
- **Initial coverage plan:** Unit test line coverage target of $\geq 80\%$ for all four high-risk services; Lighthouse performance score target of ≥ 90 .
- **Estimated testing effort:** Approximately 24 person-hours.

3 Conclusion

This assignment established the foundational QA strategy for GAN-Front, an Angular 18 SSR application developed as part of the dissertation on intelligent forensic platforms based on GAN models.

A structured risk assessment was conducted across all major modules, resulting in the identification of four high-risk components. These modules were prioritized for testing due to their high impact on system correctness, security, and end-user performance.

The QA environment was configured with a dedicated GitHub repository and a complementary toolchain. This setup ensures automated quality feedback on every code change.

The Initial Test Strategy documented above defines a clear project scope, a risk-based multi-level test approach, and specific coverage and performance targets. Baseline metrics were established to serve as reference points for the research paper.

Going forward, these metrics will be measured and reported as the test suites are implemented.